SCHOOL OF COMPUTER SCIENCE
FACULTY OF INNOVATION & TECHNOLOGY
TAYLOR'S UNIVERSITY

ITS61504 DATA MINING
ASSIGNMENT 1

SEPTEMBER 2025

LECTURER NAME: TS. DR. HUSNA SARIRAH HUSIN

STUDENT NAME:	**Angelica Julie Herliman**
STUDENT ID:	0376752
TUTORIAL GROUP:	Class 01 Section 01

Table Of Contents

Introduction

This analysis examines the development of trading and purchasing strategies as well as analysing customer purchasing patterns using Market Basket Analysis (MBA). MBA is one of the data mining techniques used to find patterns of relationships between products or goods that are frequently purchased together by customers in a single transaction. If many customers buy the same items, it can be concluded that these products have a purchasing relationship. There are many functions in using MBA, including finding customer purchasing patterns that frequently appear together, supporting marketing and promotion strategies, improving understanding of customer behaviour, optimizing how products or menus are arranged, increasing sales and stock efficiency, and serving as the foundation for systems that recommend products.

MBA is part of Association Rule Mining (ARM), which is the process of finding relationships between variables in large datasets. MBA analyses sets of customer transaction data, where each transaction contains several items purchased together. The basic concept of MBA is that past transactions can be used as a guideline or reflection of how customer behaviour will be in the future. From this, relationships between items (association rules) can be found, which can be expressed in the form of "if a customer buys product A, then it is likely that they also buy product B."

MBA uses three main measures (metrics) to assess how strong the relationships between items are:

- **Support:** shows how often a combination of items appears in all transactions. Support functions to indicate the level of popularity of an itemset.
- **Confidence:** shows how likely it is that a customer will buy B after buying A. This is done by measuring the strength of the relationship $A \rightarrow B$.
- **Lift:** shows how strong the relationship between A and B is compared to if both appeared randomly. Lift determines whether the relationship is significant or coincidental.

This analysis, conducted using MBA, aims to find association patterns between items in transaction datasets, to analyse customer purchasing behaviour based on combinations of products that are often purchased together in a single transaction, and to transform raw transaction data into meaningful insights for business decision-making.

This analysis also identifies combinations of items (frequent itemset) that often appear together in one transaction, determines relationships or associations between items, measures the strength and reliability of the relationships between items using metrics, provides information that can be used by management to determine promotional strategies and sales planning, and reveals hidden patterns in customer behaviour that are not visible through direct observation.

With this analysis, it is expected to produce association rules that clearly and measurably describe the relationships between items, provide a deep understanding of the relationships between products based on historical transaction data, and provide an analytical foundation for more effective and data-driven business strategies.

In further analysis, we can use a method called Apriori. Apriori is one of the main algorithms in association rule mining. Apriori is often used to find frequent itemset or combinations of items that frequently appear in transaction datasets. This algorithm is the most classic and popular method in the implementation of MBA. Apriori works on the principle that "if a combination frequently appears, then all of its subsets also frequently appear." Apriori uses a bottom-up iterative search approach, which means starting from small item combinations and then building up to larger combinations, while combinations that do not meet the support threshold will be removed (pruning).

Apriori is used in this analysis because it is relevant to MBA, where MBA focuses on finding patterns of relationships between items in transactions, making it very compatible with the concept of Apriori. Apriori is easy to use for data in the form of customer transaction lists, making it suitable for trade and purchase analysis. Apriori provides results in the form of association rules that are easy to understand in business contexts. Apriori is suitable for small to medium-scale datasets, and its results remain accurate and easy to process. Computationally, Apriori is supported by many Python libraries, making it easy to implement without having to build the algorithm from scratch.

Understanding of Domain and Business Problem

This analysis uses the supermarket industry as its domain. The focus of this analysis is to observe customer purchasing behavior and the relationship patterns between products in grocery purchase transactions. The grocery industry sells various customer needs, ranging from food, household necessities, beverages, and others. Customers in supermarkets often buy more than one product in a single transaction, purchases are usually made in large quantities for monthly or weekly shopping. Sales patterns in this industry are heavily influenced by various factors such as customer preferences, customer needs, product layout, as well as certain seasons or trends. The value of purchases in grocery stores is relatively large per transaction, resulting in a varied volume of transaction data. Customer loyalty and satisfaction levels toward supermarkets are influenced by the variety and completeness of goods, bundling promotions, and attractive new products.

The occurrence of many transactions every day generates a large amount of transaction data that can be used by businesses to analyse and discover relationships between goods that are often purchased together by customers. By understanding the associations between products, management can reorganize business strategies such as designing bundle packages, cross-promotion strategies, and more effective product recommendations. The MBA technique is highly relevant to this industry domain because it can identify product combinations that frequently appear as well as hidden patterns in customer purchasing behavior. The application of MBA analysis in this industry helps supermarkets make data-driven decisions to increase sales, optimize inventory to maximize sales (reducing losses), and improve the customer experience. The supermarket business in this era has intense competition, so it is important to utilize insights from transaction data to become superior and more competitive.

In the business world, there are several problems faced by management, including in the supermarket business. This business often faces high competition. Many supermarkets offer products with similar or even superior and more unique concepts. With this competition, management often struggles to understand customer purchasing behavior, especially regarding which products are frequently purchased together by customers. Supermarkets have many essential items, but management often does not know which product combinations are most preferred by customers. Business strategies such as promotions and general arrangements are often made based on assumptions, so the results are not optimal, and there is no understanding of customer purchasing patterns.

The problems above cause several issues that must be immediately addressed to optimize operations. The problems faced include a decrease in the effectiveness of promotions and bundling because they are not based on actual purchase data, lost sales opportunities due to inappropriate cross-selling strategies, inefficient stock management resulting in unsold inventory, as well as decreased customer satisfaction and unmaximized profits.

To overcome these problems, Market Basket Analysis can be applied because MBA identifies combinations of items most frequently purchased together, can create bundle promotions that match customer behavior, optimize product layout, enable data-driven decision-making instead of assumptions, increase sales and loyalty, and assist in stock and inventory planning.

The main business objective of this analysis is to identify combinations of products that are frequently purchased together by supermarket customers. Management will understand the relationship patterns between items in one transaction so that they can better understand customer purchasing behavior. Supporting data-driven decision-making in marketing and promotional strategies, the results of this analysis will later be used to design cross-selling, up-selling, and combo package strategies that are appropriate. It also aims to improve operational efficiency and sales performance through the utilization of purchasing patterns. The information on product association patterns helps manage stock, organize product layout, and increase revenue through more effective sales strategies.

Data Collection and Preprocessing

Data Collection

For this analysis, grocery data originating from supermarket transactions was used. The data was taken from Kaggle by Rashik Rahman. This grocery data shows retail product sales transactions with around 9,835 transactions. The grocery data used has three main attributes, namely Member_number, which represents the customer ID, Date, and ItemDescription, which represents the list of products purchased in one transaction. This data is transactional and categorical, containing combinations of products or itemsets that reflect customer purchasing behavior in grocery stores.

```
import pandas as pd
import numpy as np
from apyori import apriori
from sklearn.preprocessing import StandardScaler, LabelEncoder

      # Section 1: Data Collection
      transactions_df = pd.read_csv('Groceries_dataset.csv')
      print("Section 1: Data Collection")
      print("=" * 50)
      print("Displaying the first 10 rows of the raw data:")
      print(transactions_df.head(10).to_string(index=False))
```

```
Section 1: Data Collection
==================================================
Displaying the first 10 rows of the raw data:
 Member_number       Date  itemDescription  year  month  day  day_of_week
          1808 2015-07-21   tropical fruit  2015      7   21            1
          2552 2015-05-01       whole milk  2015      5    1            4
          2300 2015-09-19        pip fruit  2015      9   19            5
          1187 2015-12-12 other vegetables  2015     12   12            5
          3037 2015-01-02       whole milk  2015      1    2            4
          4941 2015-02-14       rolls/buns  2015      2   14            5
          4501 2015-08-05 other vegetables  2015      8    5            2
          3803 2015-12-23       pot plants  2015     12   23            2
          2762 2015-03-20       whole milk  2015      3   20            4
          4119 2015-12-02   tropical fruit  2015     12    2            2
```

The obtained dataset was then analysed and understood to determine the initial overview of the raw data in order to identify its deficiencies and transform it into higher-quality data. The dataset, named Groceries_dataset.csv, was read using the command pandas.read_csv(). Then, the dataset was stored into a DataFrame using Pandas with the name transactions_df. Since the data contains approximately 9,835 rows, the first 10 rows were displayed to ensure that the data was read correctly.

Data Pre-processing

The data used is not yet clean, to perform Apriori with maximum accuracy, clean data is required. This can be addressed by performing pre-processing. From the dataset, it can be seen that there are several duplicate records that must be identified and removed from the data. It is necessary to standardize the data by converting column names to lowercase and removing extra spaces, this standardization is done to avoid inconsistencies when calling columns. Transaction transformation needs to be performed because mlxtend's apriori requires one-hot encoded transactional data instead of a raw list-of-lists. Finally, a simple validation needs to be performed by checking whether the transaction_list is not empty and determining the number/maximum items per transaction.

```
# Section 2: Data Preprocessing
transactions_df.columns = transactions_df.columns.str.lower().str.strip()
print("Section 2: Data Preprocessing (standardize column names and remove duplicates)")
list(transactions_df.columns)
```

```
Section 2: Data Preprocessing (standardize column names and remove duplicates)
['member_number',
 'date',
 'itemdescription',
 'year',
 'month',
 'day',
 'day_of_week']
```

The names in the data are still messy. There are names with different writing styles, some in lowercase, some in uppercase, and some with inconsistent spacing. Therefore, column name standardization was performed, where all names were changed to one format, namely lowercase *using str.lower()* to make them more consistent and easier to call when needed. For spacing, removal was performed using *str.strip()* to prevent errors when accessing rows.

```
# Removes Duplicates
transactions_df.drop_duplicates(inplace=True)
transactions_df.reset_index(drop=True, inplace= True)
print("Removed Duplicates:")
transactions_df
```

Removed Duplicates:

	member_number	date	itemdescription	year	month	day	day_of_week
0	1808	2015-07-21	tropical fruit	2015	7	21	1
1	2552	2015-05-01	whole milk	2015	5	1	4
2	2300	2015-09-19	pip fruit	2015	9	19	5
3	1187	2015-12-12	other vegetables	2015	12	12	5
4	3037	2015-01-02	whole milk	2015	1	2	4
...	...	...	...	...	...	...	...
38001	4471	2014-08-10	sliced cheese	2014	8	10	6
38002	2022	2014-02-23	candy	2014	2	23	6
38003	1097	2014-04-16	cake bar	2014	4	16	2
38004	1510	2014-03-12	fruit/vegetable juice	2014	3	12	2
38005	1521	2014-12-26	cat food	2014	12	26	4

38006 rows × 7 columns

There are many duplicated data entries, which will greatly interfere with the analysis results because double counting will occur during the analysis. It is important to remove duplicate or unnecessary data. To address this issue, the comman *drop_duplicates(inplace=True)* can be used. This command will delete all rows that have identical or the same values. After deletion, there will be empty rows resulting from the removed duplicate rows. Therefore, to keep the data structured and tidy, and to help the data reading model, *reset_index(drop=True, inplace=True)* can be used to clean up the data.

```
grouped = transactions_df.groupby('member_number')['itemdescription']
grouped_list = grouped.agg(list).reset_index()
print("Grouped Transactions:")
grouped_list
```

```
Grouped Transactions:
```

	member_number	itemdescription
0	1000	[soda, canned beer, sausage, sausage, whole mi...
1	1001	[frankfurter, frankfurter, beef, sausage, whol...
2	1002	[tropical fruit, butter milk, butter, frozen v...
3	1003	[sausage, root vegetables, rolls/buns, deterge...
4	1004	[other vegetables, pip fruit, root vegetables,...
...	...	...
3893	4996	[dessert, salty snack, rolls/buns, misc. bever...
3894	4997	[tropical fruit, white wine, whole milk, curd,...
3895	4998	[rolls/buns, curd]
3896	4999	[bottled water, butter milk, tropical fruit, b...
3897	5000	[soda, bottled beer, fruit/vegetable juice, ro...

3898 rows × 2 columns

It will be easier to analyse the data in categorical form, therefore, the transaction data is converted into a format of grouped items per customer using this code. Converting data into grouped form is crucial because the transactions must first be organized per customer before being transformed into a one-hot encoded matrix. The Apriori function from mlxtend cannot read a raw list of items, it requires a binary one-hot encoded DataFrame where each column represents an item and each row represents a transaction. With this code, each row represents a unique transaction containing a list of all items purchased by one customer.

```
transactions_list = [grouped_list['itemdescription'][i] for i in range(len(grouped_list))]
print(f"Total transactions prepared: {len(transactions_list)}")
transactions_list[:5]
```

```
Total transactions prepared: 3898
[['soda',
  'canned beer',
  'sausage',
  'sausage',
  'whole milk',
  'whole milk',
  'pickled vegetables',
  'misc. beverages',
  'semi-finished bread',
  'hygiene articles',
  'yogurt',
  'pastry',
  'salty snack'],
 ['frankfurter',
  'frankfurter',
  'beef',
  'sausage',
  'whole milk',
  'soda',
  'curd',
  'white bread',
  'whole milk',
  'soda',
  'whipped/sour cream',
  'rolls/buns'],
 ['tropical fruit',
  'butter milk',
  'butter',
```

After grouping, the data must be transformed into a one-hot encoded matrix using TransactionEncoder. The transaction list is only an intermediate step to perform encoding, Apriori from mlxtend does not operate on a list-of-lists, but on the resulting binary DataFrame. Each ItemDescription row from the grouped results in grouped_list is converted into a list containing the names of products purchased per customer, where all these transaction lists are collected into a single variable named transaction_list.

```
te = TransactionEncoder()
te_array = te.fit(transactions_list).transform(transactions_list)
df_encoded = pd.DataFrame(te_array, columns=te.columns_)
df_encoded
```

	Instant food products	UHT-milk	abrasive cleaner	artif. sweetener	baby cosmetics	bags	baking powder	bathroom cleaner	beef	berries	...	turkey	vinegar	waffles	whipped/sour cream	whisky	white bread	white wine	whole milk	yogurt	zwieback
0	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	True	True	False
1	False	False	False	False	False	False	False	False	True	False	...	False	False	False	True	False	True	False	True	False	False
2	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	True	False	False
3	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	True	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
3893	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
3894	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	True	True	False	False
3895	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
3896	False	False	False	False	False	False	False	False	False	True	...	False	False	False	True	False	False	False	False	True	False
3897	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False

3898 rows × 167 columns

This code converts the data into a one-hot encoded matrix using the transaction encoding process. The code produces a table in which each row represents a single transaction and each column represents an item type. The values in this table are in the form of True or False, where True means the item is present in the transaction and False means the item is not present. This step is required to implement Apriori using the mlxtend library because Apriori only accepts input in binary table format.

Methodology

Association Rule Mining (ARM) is a data mining technique used to discover interesting relationships or frequently occurring patterns between items in large datasets, which will later produce "if-then" rules such as "if a customer buys product A, then they will also buy product B." Apriori is an algorithm in ARM that uses the principle of "frequent itemset generation," meaning the algorithm searches for combinations of items that frequently appear together in a single transaction. Apriori operates on the principle that if an itemset appears frequently, then all of its subsets also appear frequently. This algorithm also uses a "bottom-up" principle, starting from smaller combinations and expanding to larger ones (beginning from the smallest combinations like 1-itemsets and then forming larger item combinations from itemsets that previously passed the minimum threshold). Apriori combines this bottom-up process with "pruning," which removes itemsets that do not meet requirements such as minimum support, in order to reduce computation and improve efficiency.

With the purpose and working mechanism of apriori, it is suitable for analyzing transactional data. For example, apriori is used to analyze retail or e-commerce datasets because customers usually purchase more than one item in a single transaction. The results of Apriori analysis are also easy to interpret because the produced rules are simple, for instance, "if customers buy product A, then they are likely to buy product B." These results are very suitable for business applications because they can be used to provide insights on product arrangement based on customer behavior to increase sales. Apriori is highly effective for Market Basket Analysis because its main purpose is to discover combinations of products that are frequently purchased together.

Apriori, which is ideal for analyzing transactional or categorical data, in the mlxtend library requires data in the form of a one-hot encoded matrix (binary DataFrame). So transaction data needs to be grouped per customer first, then converted into a one-hot encoded matrix so that it can be processed by Apriori on mlxtend. By analyzing data using apriori, the expected output is in the form of frequent itemsets and association rules that describe consumer purchasing patterns.

Apriori aims to identify combinations of products that frequently appear together in asingle transaction. This allows business management to understand how customers behave during shopping and use this as a reference for customer preferences and needs. Apriori is very useful in business because it supports data-driven decision-making, meaning decisions are not based on assumptions that often lead to failure. Apriori also produces association rules with three metrics: support (how often the combination appears), confidence (how likely customers buy item B after purchasing item A), and lift (how strong the relationship between items is).

To use Apriori, clean data is required. Therefore, if the data set is still raw or dirty, preprocessing is required to convert transactions into a one-hot encoded format, where the data is a binary table (containing True/False or 1/0). After the data has been successfully encoded into a one-hot encoded DataFrame, the apriori function of mlxtend can be run to find frequent itemsets. In this analysis, a minimum support of 0.3%, minimum confidence of 50%, and minimum lift of 3 are used.

Apriori is very easy to understand and implement, making it straightforward for business managers to analyze. The produced rules are interpretable and can be readily translated into business decision-making and strategies. Apriori is supported by many Python libraries such mlxtend, which offer strong functionality and community support in frequent pattern mining.

However, despite these advantages, apriori requires long computation time when analyzing large datasets because it must calculate all possible item combinations. Apriori also requires appropriate threshold parameters (support and confidence) to ensure useful results (not too many or too few), as low thresholds may generate many irrelevant rules and lead to inaccurate analysis outputs. Additionally, apriori does not consider temporal order or purchase frequency over time, making it unsuitable for time-dependent data or strategies.

Analysis and Insights

```
print("Section 3: Apriori (mlxtend)")

min_support = 0.003
min_confidence = 0.5
min_lift = 3
min_length = 1
max_length = 4

try:
    df_encoded
except NameError:
    raise NameError("df_encoded not found. Run the one-hot encoding block (Section 2b) first.")

frequent_itemsets = mlxtend_apriori(df_encoded, min_support=min_support, use_colnames=True)

frequent_itemsets['itemset_len'] = frequent_itemsets['itemsets'].apply(lambda x: len(x))

frequent_itemsets = frequent_itemsets[
    (frequent_itemsets['itemset_len'] >= min_length) &
    (frequent_itemsets['itemset_len'] <= max_length)
].reset_index(drop=True)

if frequent_itemsets.empty:
    print("No frequent itemsets found for min_support =", min_support)
    rules_df = pd.DataFrame(columns=["antecedents","consequents","support","confidence","lift"])
else:
    rules_df = association_rules(frequent_itemsets, metric="confidence", min_threshold=min_confidence)
    rules_df['antecedent_len'] = rules_df['antecedents'].apply(lambda x: len(x))
    rules_df['consequent_len'] = rules_df['consequents'].apply(lambda x: len(x))
    rules_df = rules_df[
        (rules_df['lift'] >= min_lift) &
        (rules_df['antecedent_len'] >= min_length) &
        (rules_df['antecedent_len'] <= max_length)
    ].reset_index(drop=True)

def inspect_mlxtend(rules):
    if rules.empty:
        return []
    lhs = rules["antecedents"].apply(lambda x: tuple(sorted(x))).tolist()
    rhs = rules["consequents"].apply(lambda x: tuple(sorted(x))).tolist()
    support = rules["support"].tolist()
    confidence = rules["confidence"].tolist()
    lift = rules["lift"].tolist()
    return list(zip(lhs, rhs, support, confidence, lift))

result_df = pd.DataFrame(inspect_mlxtend(rules_df),
                         columns=["left_hand_side", "right_hand_side", "support", "confidence", "lift"])
result_df = result_df.sort_values(by="lift", ascending=False).reset_index(drop=True)

display(result_df)
```

```
Section 3: Apriori
```

	left_hand_side	right_hand_side	support	confidence	lift
0	(sliced cheese, waffles)	(yogurt, whole milk)	0.003079	0.800000	5.312436
1	(hamburger meat, waffles)	(other vegetables, soda)	0.003335	0.541667	4.362431
2	(domestic eggs, brown bread, root vegetables)	(pastry,)	0.003335	0.684211	3.854122
3	(pip fruit, waffles, other vegetables)	(pork,)	0.003079	0.500000	3.777132
4	(dessert, red/blush wine)	(pastry,)	0.003592	0.666667	3.755299
5	(sliced cheese, beef)	(yogurt, whole milk)	0.003335	0.565217	3.753352
6	(sugar, cat food)	(whole milk, other vegetables)	0.003079	0.705882	3.688377
7	(yogurt, sugar, newspapers)	(sausage,)	0.003079	0.750000	3.640722
8	(whipped/sour cream, condensed milk)	(sausage,)	0.003079	0.750000	3.640722
9	(grapes, citrus fruit, other vegetables)	(frankfurter,)	0.003079	0.500000	3.636194
10	(sausage, bottled water, waffles)	(bottled beer,)	0.003079	0.571429	3.598431
11	(oil, beverages)	(rolls/buns, other vegetables)	0.003079	0.521739	3.555488
12	(other vegetables, beef, coffee)	(whipped/sour cream,)	0.003079	0.545455	3.526006
13	(dessert, white bread)	(rolls/buns, other vegetables)	0.004874	0.500000	3.407343
14	(white bread, bottled beer, soda)	(whipped/sour cream,)	0.003335	0.520000	3.361459
15	(hamburger meat, oil)	(yogurt, whole milk)	0.003079	0.500000	3.320273
16	(soft cheese, sausage)	(yogurt, whole milk)	0.003335	0.500000	3.320273
17	(shopping bags, whole milk, salty snack)	(canned beer,)	0.004361	0.548387	3.319275
18	(hamburger meat, waffles)	(whole milk, soda)	0.003079	0.500000	3.308998
19	(baking powder, chocolate)	(bottled water,)	0.003079	0.705882	3.303157
20	(sliced cheese, waffles)	(yogurt,)	0.003592	0.933333	3.298398

In its implementation, apriori() and association_rules() from the mlxtend library are used. When using Apriori for analysis, we must first determine the parameters that will be used as the required standards according to the desired results. In this code, 0.003 is used for minimum support, meaning that only itemsets appearing in at least 0.3% of the transactions are considered. 0.5 is used for confidence, meaning the minimum accepted certainty level is 50%. The minimum lift is set to be greater than or equal to 3, which means the selected items must have a strong relationship. It is also specified that each transaction must contain a minimum of 1 item and a maximum of 4 items.

The algorithm searches for frequent itemsets that generate association rules based on these parameters. The resulting output is then converted into a table format to make it easier to read. The DataFrame containing the Left Hand Side ("If buying"), Right Hand Side ("then will buy"), support, confidence, and lift is automatically generated using association_rules() from the mlxtend library. The DataFrame is then created and sorted from the highest lift to the lowest lift, placing the strongest relationships at the top.

Threshold set	Min Support	Min Confidence	Min lift	Total Rules
1	0.003	0.5	3	44 Rules
2	0.002	0.55	4	57 Rules
3	0.002	0.6	4	39 Rules
4	0.002	0.5	2	3534 Rules

Several experiments were conducted using different thresholds. Based on the experiment, the combination of minimal support = 0.003, minimal confidence = 0.5, and minimal lift = 3 was selected as the optimal threshold. This threshold produces a balanced number of rules at 44 rules, which is neither too high nor too low. A lower minimum threshold generates too many rules, making them too overwhelming to analyse, while a threshold that is too high results in too few rules to analyse, causing management to miss potential rules that could be useful for decision-making.

From these experiments, it can be concluded that the threshold value greatly affects the number of association rules generated by the algorithm. Differences in thresholds filter whether a rule is considered strong and relevant enough to be analysed. From this table, we can conclude that the stricter the threshold, the fewer the rules, conversely, the more lenient the threshold, the more rules will be generated. In addition, combinations of thresholds also produce different numbers of rules, making it very important to choose balanced parameters.

The principles and formulas of Apriori are very important to understand when using it because from here we can determine the relationship and how strong the relationship is between the items.

- Support: measures how often an itemset appears in all transactions.
  Support has the following formula:

  $$\text{Support}(A) = \frac{\text{Number of transactions containing } A}{\text{Total number of transactions}}$$

  From this, we can see that the higher the support, the more frequently the combination of items is purchased.
- Confidence: measures the probability that customers will buy item B after purchasing item A.
  The formula used is:

  $$\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

  From the formula, it can be concluded that high confidence indicates a high probability for the rule “if buying A then also buying B.”
- Lift: measures the strength of the relationship between items compared to random occurrence.
  Formula:

$$\text{Lift}(A \rightarrow B) = \frac{\text{Confidence}(A \rightarrow B)}{\text{Support}(B)}$$

Or

$$\text{Lift}(A \cup B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A) \times \text{Support}(B)}$$

Where:

- Lift > 1 : the relationship is positive
- Lift = 1 : there is no relationship
- Lift < 1 : the relationship is negative

- Frequent itemset principle: contains the fundamental principle of Apriori which states, "If an itemset is large (frequent), then all of its subsets are also frequent." This principle means the algorithm does not need to evaluate larger combinations whose subsets do not meet the minimum support.
- Bottom-up approach: Apriori builds itemsets from the smallest, starting from 1-itemset up to n-itemset.
- Rule generation: if a frequent itemset is found, rules will be formed:
  $A \rightarrow B$
  These rules are built by calculating the support rule, confidence rule, lift rule, and filtered based on the thresholds determined according to the needs of the analysis.

From the results of the analysis performed using the Apriori algorithm, a table was obtained containing the Left Hand Side and Right Hand Side along with their support, confidence, and lift values. The results of the analysis are also sorted by the highest lift, so the table shows the order of items with the strongest influence. For example, sliced cheese and waffles have the strongest influence with yogurt and whole milk, with a support of 0.003079, confidence of 0.80, and lift of 5.31. This indicates a high likelihood that customers will buy yogurt and whole milk when they purchase sliced cheese and waffles.

Rule	Support	Antercedent Support	Consequent Support	Leverage
(Dessert, red/blush wine) → pastry	0.00359	0.00539	0.17753	0.00264
(Sliced Cheese, waffles) → (yogurt, whole milk)	0.00359	0.00385	0.28297	0.00265
(Ham, whipped/sour cream) → condensed milk	0.00333	0.00462	0.20967	0.00236
(Condensed milk, whipped/sour cream) → sausage	0.00308	0.00410	0.20600	0.00223
(Baking powder, chocolate) → bottled water	0.00308	00.436	0.21370	0.00215

Leverage analysis was conducted to identify truly significant purchasing patterns within the data, not coincidental ones. This analysis identified five rules with the highest leverage values. The table above concludes that positive and high leverage indicates a strong and relevant relationship between products. This can be used to optimize business strategies. It can be concluded that

positive and high leverage values demonstrate strong and meaningful relationships between products. This information can be used to optimize business strategies.

From this pattern, business management can derive several business insights:

- The creation of breakfast packages containing waffles, sliced cheese, yogurt, and whole milk, since these items are often purchased together, forming a shopping pattern categorized as breakfast essentials.
- Combinations of meat products such as hamburger meat and sausage, vegetables, and soda indicate a habit among customers who prefer purchasing ready-to-cook meals.
- There are also several snack or weekend treat products that are frequently bought together, such as pastries, desserts, and red/blush wine.

From these observations, we can see that clusters are formed during shopping, such as:

- breakfast cluster: dairy + pastry/waffles + yogurt
- meal-prep cluster: meat + vegetables + beverages
- snacking cluster: pastry + dessert

From these insights, business management can transform them into business strategies that can be used to optimize operations:

1. Cross-selling or direct recommendations in-store/website
   When a customer buys product A, product B can be recommended based on business insights. For example, when a customer buys waffles, yogurt and whole milk can be recommended.
2. Product bundling or sales packages
   Create bundle packages where, when a customer buys product A, it can be bundled with products B and C at a lower price. For example, offering a breakfast bundle containing waffles, sliced cheese, yogurt, and whole milk. This encourages customers to buy because they can get their breakfast needs all at once without searching in separate sections, and the price is cheaper.
3. Store layout optimization
   Using the insights, products can be arranged more effectively. Products can be grouped according to lift values, the highest lift products can be placed near each other physically. For example, dairy can be placed near the pastry/waffle section, so when customers buy pastries, they are more likely to also buy dairy due to the strong lift relationship.
4. Promo and discount targeting
   Management can offer small discounts on Right Hand Side products. For example, each purchase of waffles offers a 5% discount on yogurt.
5. Inventory management
   Using Apriori, management can determine appropriate stock levels for each product. Products that are frequently bought together can have increased stock, while products that are rarely purchased can have reduced stock. This can be observed by looking at support.

From this, we can see that Apriori generates strong and valid rules for the data, based on support, confidence, and lift. A high lift (greater than or equal to 3) indicates that items are not purchased randomly but reflect actual customer behavior in transactions. The Apriori results are highly useful for businesses because management becomes capable of designing business strategies to increase

sales through cross-selling, product bundling, optimizing product layout in stores, and improving stock and supply strategies. Based on the analysis conducted with Apriori, businesses can make wise, data-driven decisions rather than relying on assumptions, which is expected to increase profits and provide customers with a satisfying shopping experience.

Overall, it can be concluded that the Apriori algorithm successfully identified strong and significant purchasing patterns through support, confidence, lift, and leverage, which show that the relationships between products are not random but instead reflect consistent customer shopping behavior. The combination of thresholds used produced 44 high-quality rules that can be utilized by business management as business insights for decision-making or strategy planning. Leverage here reinforces the existence of strong relationships between products (confirming them). Items that frequently appear such as yogurt, pastry, condensed milk, sausage, and bottled water indicate that customers often purchase these products, which can also be used as a data-driven business strategy for management. Rules with high leverage can provide the most effective cross-selling and bundling opportunities because their purchasing patterns are highly consistent in the data. Therefore, this analysis provides a strong foundation for more optimal marketing strategies, improvements in product placement, and optimization of inventory levels because the associations between products are proven to be stable and statistically significant.

Visualization and Presentation Results

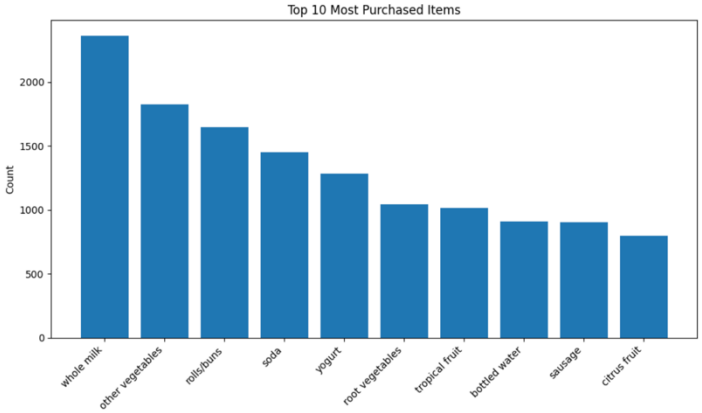
index	left_hand_side	right_hand_side	support	confidence	lift
0	sliced cheese,waffles	yogurt,whole milk	0.0030785017957927143	0.8	5.312436115843271
1	hamburger meat,waffles	other vegetables,soda	0.0033350436121087736	0.5416666666666666	4.362431129476584
2	domestic eggs,brown bread,root vegetables	pastry	0.0033350436121087736	0.6842105263157895	3.854122299969577
3	pip fruit,waffles,other vegetables	pork	0.0030785017957927143	0.5	3.7771317829457365
4	dessert,red/blush wine	pastry	0.0035915854284248334	0.6666666666666666	3.7552986512524082
5	sliced cheese,beef	yogurt,whole milk	0.0033350436121087736	0.5652173913043478	3.7533516035849193
6	sugar,cat food	whole milk,other vegetables	0.0030785017957927143	0.7058823529411765	3.68837722756663
7	yogurt,sugar,newspapers	sausage	0.0030785017957927143	0.75	3.6407222914072226
8	whipped/sour cream,condensed milk	sausage	0.0030785017957927143	0.75	3.6407222914072226
9	grapes,citrus fruit,other vegetables	frankfurter	0.0030785017957927143	0.5	3.636194029850746
10	sausage,bottled water,waffles	bottled beer	0.0030785017957927143	0.5714285714285714	3.598430648511424
11	oil,beverages	rolls/buns,other vegetables	0.0030785017957927143	0.5217391304347826	3.555487990270599
12	other vegetables,beef,coffee	whipped/sour cream	0.0030785017957927143	0.5454545454545455	3.5260063319764816
13	dessert,white bread	rolls/buns,other vegetables	0.004874294510051305	0.5	3.407342657342657
14	white bread,bottled beer,soda	whipped/sour cream	0.0033350436121087736	0.5199999999999999	3.3614593698175783
15	hamburger meat,oil	yogurt,whole milk	0.0030785017957927143	0.5	3.320272572402044
16	soft cheese,sausage	yogurt,whole milk	0.0033350436121087736	0.5	3.320272572402044
17	shopping bags,whole milk,salty snack	canned beer	0.004361210877373012	0.5483870967741935	3.3192746944500096
18	hamburger meat,waffles	whole milk,soda	0.0030785017957927143	0.5	3.3089983022071308
19	baking powder,chocolate	bottled water	0.0030785017957927143	0.7058823529411765	3.3031565567403436
20	sliced cheese,waffles	yogurt	0.0035915854284248334	0.9333333333333333	3.298398307645814
21	brown bread,soda,coffee	canned beer	0.0035915854284248334	0.5384615384615385	3.259197324414716
22	pip fruit,bottled water,berries	shopping bags	0.0030785017957927143	0.5454545454545455	3.2411308203991136
23	napkins,bottled water,shopping bags	citrus fruit	0.0030785017957927143	0.6	3.234854771784232
24	whole milk,beef,coffee	whipped/sour cream	0.0035915854284248334	0.5	3.2321724709784414
25	soft cheese,sausage	rolls/buns,whole milk	0.003848127244708927	0.576923076923077	3.2311007957559688
26	other vegetables,chocolate,oil	citrus fruit	0.0033350436121087736	0.5909090909090909	3.1858418206965924
27	chocolate,whole milk,coffee	pip fruit	0.0033350436121087736	0.5416666666666666	3.175062656641604
28	hamburger meat,misc. beverages	sausage	0.0033350436121087736	0.65	3.1552926525529266
29	specialty chocolate,bottled water,yogurt	citrus fruit	0.003848127244708927	0.576923076923077	3.110437280561762
30	chicken,whole milk,domestic eggs	shopping bags	0.003848127244708927	0.5172413793103449	3.073486122792263
31	red/blush wine,whole milk,root vegetables	pastry	0.0030785017957927143	0.5454545454545455	3.0725170782974254
32	sliced cheese,whole milk,pork	sausage	0.0030785017957927143	0.6315789473684211	3.065871403290293
33	misc. beverages,root vegetables,soda	sausage	0.0030785017957927143	0.6315789473684211	3.065871403290293

This table is the result of Apriori, containing a list of association rules generated based on customer transaction data. The table is sorted by the highest lift so that the strongest relationships appear at the top. There are 5 components in this table:

1. Left Hand Side contains the items that are purchased first by the customer.
2. Right Hand Side contains the items that are most likely purchased after the items in the LHS.
3. Support contains the proportion of transactions that include both the LHS and RHS. The support value indicates whether the rule appears frequently enough in the overall dataset.
4. Confidence shows the probability that customers buy the RHS after buying the LHS.
5. Lift shows how strong the relationship LHS → RHS is compared to if the purchase occurred randomly, where lift equal to or greater than 3 indicates a strong and significant relationship.

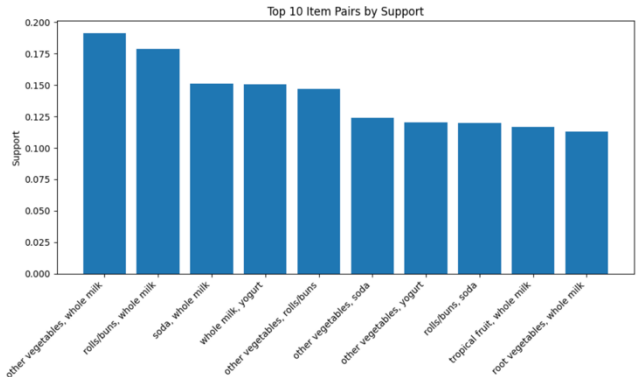
The highest lift is found in (Sliced cheese, waffles) → (yogurt, whole milk) with a lift of 5.31, meaning this relationship is 5 times stronger than random purchases. The highest confidence is around 0.84, which appears in (dessert, red wine) → (pastry), meaning 84% of customers who buy dessert + red/blush wine also buy pastry, making this transaction the most "certain to occur" in consumer behavior. And (whole milk, rolls/buns) → (other vegetables) has the highest support around 0.00487, this rule appears the most frequently compared to the others, making it the most relevant for strategies that require high volume.

```python
# Section 6: Bar chart – Top 10 Most Purchased Items
import matplotlib.pyplot as plt
from collections import Counter

all_items = [item for tx in transactions_list for item in tx]
item_counts = Counter(all_items)
top10 = item_counts.most_common(10)
top10_df = pd.DataFrame(top10, columns=['item','count'])

plt.figure(figsize=(10,6))
plt.bar(top10_df['item'], top10_df['count'])
plt.xticks(rotation=45, ha='right')
plt.ylabel('Count')
plt.title('Top 10 Most Purchased Items')
plt.tight_layout()
plt.show()

plt.savefig('section6_top10_items.png', bbox_inches='tight')
```



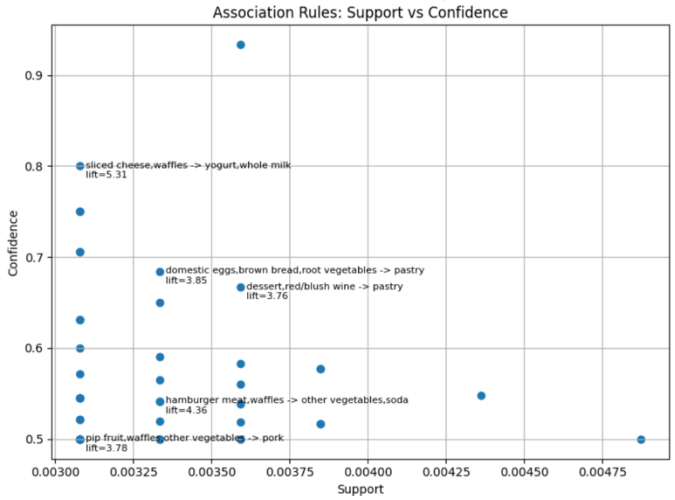
The bar chart is used to display the ten most purchased products in all transactions. This graph is arranged based on the items most frequently purchased, with whole milk located on the far left, down to the items that are not purchased as frequently within the top 10 products, namely citrus fruit. From this bar chart, we can see that the higher the bar, the greater the frequency of the item's appearance in transactions, or the more often the item is purchased by customers. This distribution shows the most dominant items and provides an important foundation for analysis before examining item combinations and performing the creation of association rules. Items with high frequencies usually have the potential to appear in many frequent itemsets, making it important to understand which items are frequently purchased by customers.

```
# Section 7: Bar chart – Top 10 Item Combinations (pairs) by support
from itertools import combinations

pair_counts = Counter()
for tx in transactions_list:
    unique_items = sorted(set(tx))
    for a,b in combinations(unique_items, 2):
        pair_counts[(a,b)] += 1

n_tx = len(transactions_list)
pairs_df = pd.DataFrame([
    {'itemA': a, 'itemB': b, 'count': cnt, 'support': cnt / n_tx}
    for (a,b), cnt in pair_counts.items()
])
pairs_df = pairs_df.sort_values('support', ascending=False).reset_index(drop=True)
top10_pairs = pairs_df.head(10).copy()
top10_pairs['pair_str'] = top10_pairs['itemA'] + ', ' + top10_pairs['itemB']

plt.figure(figsize=(10,6))
plt.bar(top10_pairs['pair_str'], top10_pairs['support'])
plt.xticks(rotation=45, ha='right')
plt.ylabel('Support')
plt.title('Top 10 Item Pairs by Support')
plt.tight_layout()
plt.show()

plt.savefig('section7_top10_itempairs.png', bbox_inches='tight')
```



This chart also uses a bar chart to show the ten combinations of two items, or pairs, that most frequently appear together in transactions. These pairs are calculated using support, so the chart shows the proportion of customers who purchased the two items simultaneously. From the bar chart, we can see the highest customer interest based on how high the bars are, the higher the bar, the greater the customer interest in that pair combination. From the chart, we can see that *other vegetables + whole milk* is the combination most frequently purchased by customers with a support of 0.19, followed by *rolls/buns + whole milk* with a support of 0.18. Conversely, at the bottom is *root vegetables + whole milk* with a support of 0.10. This chart will help management in making business decisions because it helps identify the most popular basic item combinations, common customer shopping patterns, and foundational priorities before moving into 3-item frequent itemsets or association rules.

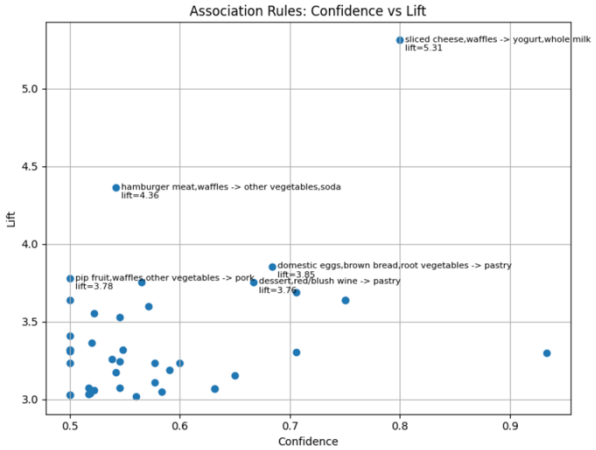
```python
# Section 8: Scatter plot – Support vs Confidence for rules found by apyori

if 'result_df' not in globals():
    raise RuntimeError("result_df not found. Run Section 3 (Apriori) first.")

plt.figure(figsize=(8,6))
plt.scatter(result_df['support'], result_df['confidence'])
plt.xlabel('Support')
plt.ylabel('Confidence')
plt.title('Association Rules: Support vs Confidence')
plt.grid(True)

try:
    top5 = result_df.sort_values('lift', ascending=False).head(5)
    for _, row in top5.iterrows():
        lhs = ','.join(row['left_hand_side']) if isinstance(row['left_hand_side'], (list, tuple)) else str(row['left_hand_side'])
        rhs = ','.join(row['right_hand_side']) if isinstance(row['right_hand_side'], (list, tuple)) else str(row['right_hand_side'])
        label = f"{lhs} -> {rhs}\nlift={row['lift']:.2f}"
        plt.annotate(label, (row['support'], row['confidence']), textcoords="offset points", xytext=(5,-10), fontsize=8)
except Exception as e:
    print("Annotate skipped:", e)

plt.tight_layout()
plt.show()

plt.savefig('section8_support_vs_confidence.png', bbox_inches='tight')
```



A scatter plot is used to show the relationship between variables. In this case, it is used to observe the relationship between support and confidence. This scatter plot displays each association rule as a point, with the X-axis representing support, which shows how often the rule appears (the farther to the right, the more frequently it occurs), and the Y-axis representing confidence, which shows how likely customers are to buy the RHS after buying the LHS (the higher the point, the more reliable the rule is).

This chart helps differentiate rules that occur rarely but are strong (rules with low support but high confidence), rules that occur frequently but are not very strong (rules with high support but moderate confidence), and the best rules (ideal rules located in the upper-right area with both high support and confidence). From the chart, it can be seen that (dessert + red/blush wine) → pastry has the highest confidence, and (whole milk + rolls/buns) → other vegetables has the highest support.

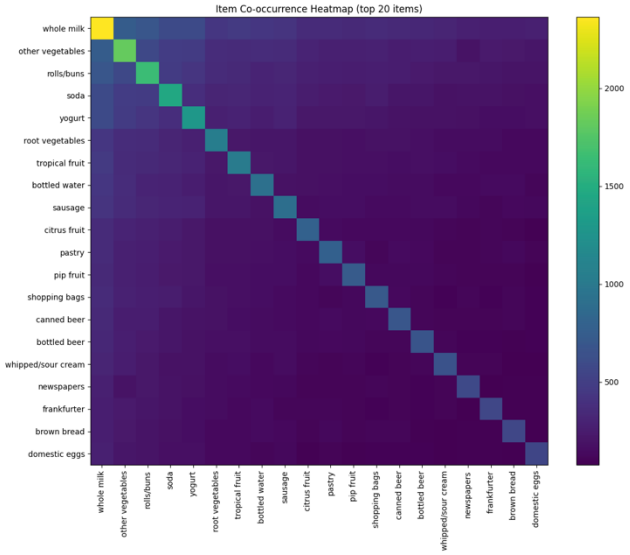
```python
# Section 9: Scatter plot - Confidence vs Lift for rules found by apyori
import matplotlib.pyplot as plt

if 'result_df' not in globals():
    raise RuntimeError("rResult_df not found. Run Section 3 (Apriori) first.")

plt.figure(figsize=(8,6))
plt.scatter(result_df['confidence'], result_df['lift'])
plt.xlabel('Confidence')
plt.ylabel('Lift')
plt.title('Association Rules: Confidence vs Lift')
plt.grid(True)

try:
    top5_lift = result_df.sort_values('lift', ascending=False).head(5)
    for _, row in top5_lift.iterrows():
        lhs = ','.join(row['left_hand_side']) if isinstance(row['left_hand_side'], (list, tuple)) else str(row['left_hand_side'])
        rhs = ','.join(row['right_hand_side']) if isinstance(row['right_hand_side'], (list, tuple)) else str(row['right_hand_side'])
        label = f"{lhs} -> {rhs}\nlift={row['lift']:.2f}"
        plt.annotate(label, (row['confidence'], row['lift']), textcoords="offset points", xytext=(5,-10), fontsize=8)
except Exception as e:
    print("Annotate skipped:", e)

plt.tight_layout()
plt.show()

plt.savefig('section9_confidence_vs_lift.png', bbox_inches='tight')
```



To achieve the most optimal business strategy decision-making results, an analysis of the relationship between confidence and lift can be performed using a scatter plot. The metrics used in this chart are confidence as the Y-axis, which measures predictive strength (how likely customers are to buy the RHS after buying the LHS), and lift as the X-axis (which measures the strength of the relationship between the LHS and RHS compared to if the purchase occurred randomly). This scatter plot is able to identify rules that are very strong and highly reliable for consideration in business decision-making.

This scatter plot can be interpreted as follows: the farther to the right the point is, the higher the confidence, the higher the point, the greater the lift. Therefore, points located in the upper-right area represent rules with the best quality. From the chart, it can be seen that (sliced cheeses, waffles) → (yogurt, whole milk) has the highest lift with 5.31, and (whole milk, yogurt) → (pastry) has the highest confidence at around 0.93–0.94. It can be concluded from the chart that the most prominent rule in overall quality is (sliced cheese, waffles) → (yogurt + whole milk) because it has the highest lift and strong confidence.

```python
# Section 10: Heatmap – Item Co-occurrence (top N items)
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from itertools import combinations

if 'transactions_list' not in globals():
    raise RuntimeError("transactions_list not found. Run Section 2 (preprocessing) first.")

TOP_N = 20
all_items = [item for tx in transactions_list for item in tx]
item_counts = Counter(all_items)
top_items = [it for it,_ in item_counts.most_common(TOP_N)]

idx = {it:i for i,it in enumerate(top_items)}
mat = np.zeros((len(top_items), len(top_items)), dtype=int)

for tx in transactions_list:
    items = set(tx) & set(top_items)
    for a,b in combinations(items, 2):
        i,j = idx[a], idx[b]
        mat[i,j] += 1
        mat[j,i] += 1

for it,i in idx.items():
    mat[i,i] = item_counts[it]

plt.figure(figsize=(12,10))
plt.imshow(mat, interpolation='nearest', aspect='auto')
plt.colorbar()
plt.xticks(ticks=np.arange(len(top_items)), labels=top_items, rotation=90)
plt.yticks(ticks=np.arange(len(top_items)), labels=top_items)
plt.title(f'Item Co-occurrence Heatmap (top {len(top_items)} items)')
plt.tight_layout()
plt.show()

plt.savefig('section10_item_cooccurrence_heatmap.png', bbox_inches='tight')
```



To summarize all the results from the table, bar chart, and scatter plot, a heatmap can be used to see how often two items appear together in one transaction. This heatmap shows the 20 items that are frequently purchased by customers, where each cell represents the co-occurrence count, which is the number of transactions in which both items appear in the same transaction. The way to analyse the heatmap is by looking at the colour of each cell. Yellow or bright colours indicate high co-occurrence, while purple or dark colours indicate low co-occurrence. The main diagonal on the heatmap appears brighter because it represents the frequency of each individual item. The heatmap helps identify general patterns using pairwise co-purchasing.

From the heatmap, we can see that whole milk has the highest co-occurrence with many other items purchased by customers such as other vegetables, rolls/buns, soda, and others. The results from the heatmap reinforce the patterns found in the previous bar chart, where the item frequency chart shows that whole milk and other vegetables are dominant in the dataset. It also shows that (other vegetables, whole milk) and (rolls/buns, whole milk) are pairs frequently bought by customers, as well as several strong rules involving dairy and vegetables. Therefore, overall, the heatmap provides a macro visual of the co-occurrence structure within a dataset.

Conclusion and Recommendation

Apriori is one of the algorithms in Association Rule Mining (ARM) used in Market Basket Analysis (MBA) to find patterns of relationships between products in transactional or categorical data. Apriori works by identifying combinations of items that frequently appear together, making it suitable for transactional data because it can accurately detect customer shopping behavior, where customers typically purchase more than one item in a single transaction. In this research, the groceries dataset containing supermarket customer transactions is used, and this dataset aligns well with the purpose of Apriori, enabling the analysis of customer shopping behavior patterns. Apriori can provide product pairs that frequently appear together, which can then be analyzed by business management for decision-making. Before applying Apriori, it is important to perform data collection and preprocessing by standardizing column names, removing duplicates, grouping data, and converting the data into one-hot encoding format because Apriori requires input in the form of a one-hot encoded matrix.

The results of the Apriori analysis reveal several strong purchasing patterns based on support, confidence, and lift. In this analysis, the rule with the highest lift appears in the combination (sliced cheese, waffles) → (yogurt, whole milk). This combination indicates a strong relationship among the four products compared to random occurrence. The rule with the highest confidence is (dessert, red/blush wine) → pastry, showing a purchasing behavior that almost always occurs together. Lastly, the combination with the highest support is (whole milk, rolls/buns) → other vegetables, showing that these items appear most frequently in transactions and are highly relevant. These discovered patterns show that customers commonly purchase products related to needs such as breakfast, cooking ingredients, or weekend consumption.

From the patterns obtained in the analysis, these can be transformed into business recommendations that management can implement to optimize business operations. First, the store can perform targeted promotions by automatically recommending products or offering discounts on RHS products. For example, offering yogurt when a customer buys waffles or sliced cheese. Second, the store can reorganize product placement by placing products with strong associations close to each other, such as positioning rolls/buns near vegetables because they are often purchased together. Third, the store can create bundles from frequent itemsets or items frequently purchased together. For example, creating a breakfast bundle containing waffles, sliced cheese, yogurt, and whole milk, which can increase transaction value. Finally, with these patterns, businesses can adjust supply stock and ordering so that frequently purchased products receive increased stock and products less frequently purchased receive reduced stock. This shows that with Apriori, business strategies can be based on data and structured according to the association rules discovered. With Apriori analysis, it is highly expected that the store can increase its sales, optimize store layout, reduce the risk of stockouts and overstock, and achieve other business goals.

Bibliography

Apus Nest. (2025, August 31). *Market basket analysis example: A step-by-step guide*. Retrieved from https://www.apusnest.com/blog/market-basket-analysis-example

DataCamp. (2023, January 23). *Association rule mining in Python tutorial*. Retrieved from https://www.datacamp.com/tutorial/association-rule-mining-python

GeeksforGeeks. (2025, September 8). *Association rule*. Retrieved from https://www.geeksforgeeks.org/machine-learning/association-rule/

Kaggle. (2020). Rahman, R. (Dataset author). *Groceries dataset for Market Basket Analysis (MBA)*. Retrieved from https://www.kaggle.com/datasets/rashikrahmanpritom/groceries-dataset-for-market-basket-analysismba

Quantzig. (n.d.). *Market Basket Analysis in Retail: An Important Tool in Any Retailer's Arsenal*. Retrieved from https://www.quantzig.com/blog/market-basket-analysis-retail-industry/

Rasbt, S. (n.d.). *Apriori: Frequent itemsets via the Apriori algorithm*. Retrieved from https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/apriori/

Sigma-NITT. (2023, August). *Market basket analysis*. Retrieved from https://sigma-nitt.medium.com/market-basket-analysis-f6f36e4e0ae3

Wikipedia. (2025, October.). *Apriori algorithm.* Retrieved from https://en.wikipedia.org/wiki/Apriori_algorithm